

2026.6.22. 统计与大数据分析软件.

By: LJ

一. 选择 (12个Python, 8个R)

Python ① 类似 $a = [1, 2, 3]$

$b = a$

... 赋值

这里反写一些能想起的考点

② $funcs = [fun1(), fun2(), fun3()]$

$print(funcs[2])$

类似前年考题

R: aes , $sapply$... 向量的选择用哪个函数

二. Iris 数据集. ① 分布情况

② 两列数据是否有线性关系

分别用什么图, 并解释原因.

三. R中 Vector, List, Matrix, Data frame 分别有什么特点,

它们又有什么区别 (此外回忆几乎与题干一字不差)

(最后一句, 我的上一届考了 $apply$ 系列的辨析)

四. Python 看程序写运行结果并解释原因.

Class Book:

$price = 50$

.....

先改了 $b.price$ 然后 $print b$

再改了 $Book.price$, $print b_1$ 和 b_2 .

五. 神经网络填空并解释作用:

要填的只有下面画线部分函数. 五个空.

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

3. Define a Loss function and optimizer ^^^ Let's use a Classification Cross-Entropy loss and SGD with momentum.

```
In [6]: import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

4. Train the network ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

This is when things start to get interesting. We simply have to loop over our data iterator, and feed the inputs to the network and optimize.

```

In [17]: for epoch in range(2): # Loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999: # print every 2000 mini-batches
            print('%5d, %5d) loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0

    print('Finished Training')

```